Solace: A Technical White Paper

Sep 28, 2026 · @Aaron

Summary

Solace is a programming system designed for software that AI agents build and maintain. A Solace program is stored as a database of symbols (functions, types, constants and tests), each with a permanent ID, a type-checked definition and a record of what it depends on. Source text is generated from the database when someone needs to read it. Agents read and change the program one symbol at a time through a small set of commands, and every change is type-checked before it is saved.

Solace is at an early stage. The database and agent commands exist as a prototype that manages code written in Gleam, an existing functional language; the Solace language itself comes later.

1. The problem

Programming tools are built around people editing text files. An agent working that way spends much of its effort on things unrelated to the task:

- **Finding its way around.** It searches, opens and reads files for several rounds before changing anything, and it re-sends its whole conversation to the model on every step.
- **Editing text.** Find-and-replace patches fail when whitespace differs or the search text appears more than once.
- **Remembering.** Each session starts with no record of earlier plans or decisions.
- **Hidden inputs.** Time, randomness and network calls don't appear in function signatures, so behavior is hard to predict and tests can be flaky.

2. The data model

Everything in a program is a symbol: a function, type, type variant, constant or test. Each symbol has:

Field	Meaning
ID	Assigned by the system and never reused, e.g. `fn:7f3a92c1e0b44d21`
Name	`module.name`
Revision	A hash of the symbol's content
Metadata	Description, verification status, history

Symbols refer to each other by ID, so renaming a function doesn't require editing its callers. Editing or renaming a symbol keeps its ID, even if the body is rewritten from scratch; a copy gets a new ID linked to the original. Deleting is refused while anything still uses the symbol, and a deleted symbol's ID is never reused.

A revision hashes what the symbol does: its signature, body, and the IDs of the symbols it uses. It leaves out the name, formatting, comments and local variable names, so renaming, documenting or reformatting a function keeps test results recorded against it valid.

Each symbol has exactly one text form, produced by a formatter with no options, so formatting never shows up in a diff. Modules are shown as outlines (signatures and one-line descriptions) unless full bodies are requested. The whole program can be exported as plain text for Git, one block per symbol with its ID in a header:

```
-- @id fn:7f3a92c1 @rev sha256:a83f...
fn calculateBuoyancy(parcel: SmokeParcel, environment: Environment) ->
Acceleration
    effects: none
```

3. How data is stored

The program lives in one SQLite file, `.solace/store.db`, in the project root. The prototype uses five tables:

```
modules(path PRIMARY KEY, dir, header)                -- module path, src or
test, imports
symbols(id PRIMARY KEY, module, name, kind,
        text, ord, revision, live DEFAULT 1)          -- one row per symbol;
deleted = live 0
history(seq PRIMARY KEY, at, symbol, action, detail)  -- every change, in
order
broken(symbol PRIMARY KEY, diagnostic)                -- symbols broken by a
saved change
work(id PRIMARY KEY, text, open DEFAULT 1)            -- notes that carry
over between sessions
```

`solace import` splits an existing Gleam project into symbols. Dependencies between symbols are worked out from their text and drive the "who uses this" queries. After each change, Solace writes the Gleam files back out for the normal compiler and test runner. Agents never see those files.

4. How the agent works

The agent gets one tool, `solace`, provided over MCP (Model Context Protocol, the standard way coding agents call external tools), and no file access or shell. The tool takes `command` (one or more command lines), `text` (new code, the task description, or a note) and `old` (text to replace, for small edits).

A typical session:

1. `session start --task`, with the task as `text`. Solace matches words in the task against names, documentation and code, adds callers, called functions and covering tests, and returns about 2,000 tokens: each symbol's signature, description and why it was chosen, plus notes from earlier sessions.
2. `inspect a b c --context` returns each symbol's code, the signatures of what it uses, its callers, and the smallest test that exercises it.
3. `submit <name>` with new code. Solace parses, type-checks and saves it in one step.
4. `check` and `test`, which report only problems.
5. `work add` leaves a note for the next session.

Command	What it does
`session start [--task]`	Starting point: relevant symbols, modules, saved notes
`inspect <name>… [--context]`	Symbols with dependencies, callers, an example test
`view <module>… [--bodies]`	Module outlines, or full source
`refs <name>` / `find <text>`	What uses a symbol / search all code
`submit <name>`	Replace a symbol
`submit <name> --replace`	Replace one exact piece of text inside a symbol
`submit --new <module>`	Add a symbol
`submit`	Several symbols at once, checked together
`delete <name>`	Remove a symbol (refused while anything uses it)
`check`, `test`	Type-check everything, run tests
`work add`, `work list`, `work done`	Notes that carry over between sessions

If submitted code has errors, nothing is saved. If it is correct but breaks other code, such as callers of a changed signature, it is saved and the broken callers are listed until fixed. Errors carry a stable code, name the symbol, state what was expected and found, and group follow-on errors under the first.

5. Advantages

- **A short path to the right code.** The task-based starting list and the `inspect` bundle replace most of the search-open-read cycle.
- **Smaller reads.** The agent reads the definitions it needs, not whole files.
- **Reliable edits.** The agent replaces a named symbol, so an edit can't fail on whitespace or change the wrong place.
- **Fast feedback.** Each change is type-checked in about half a second; reads take milliseconds.
- **Step-by-step refactors.** Broken callers are tracked, so a signature change can be fixed one caller at a time, or submitted together with its callers.

- **Less bookkeeping.** Imports, formatting and rename fix-ups are never the agent's job.
- **A narrow, auditable interface.** With one tool and no file system or shell, the agent can only work through Solace, and every change is recorded in the history.
- **Memory between sessions.** Saved notes appear at the next session start along with the code they mention.

6. Costs and open problems

- **Larger edits.** A file patch sends only changed lines; a submit usually re-sends the whole symbol. In testing, each Solace step produced about 60% more output and tasks took about 18% longer in wall-clock time. The `--replace` edit is meant to fix this but is new.
- **Unfamiliar tools.** Models are trained on file-based workflows and still open whole modules or read tests one at a time, which costs extra steps.
- **Output size matters on every step.** Anything the agent reads is re-sent each step. An early session start listed every signature (about 8,000 tokens) and cost more than it saved; it now lists names only.
- **Uneven gains.** Tasks spanning several modules benefit most; some small changes cost more than with files.
- **Friction with file-based tools.** Editors, linters and GitHub review depend on the export, which does not yet round-trip.
- **Missing pieces.** Committing changes with conflict detection, rename and merge commands, questions routed to a human, and the Solace language are not built yet.

7. Early results

The test project is an order-management service in Gleam (about 30,000 tokens of code, 98 tests) with ten tasks: new features, bug fixes, a change touching many callers, a state-machine change, and resuming half-finished work. The same agent (the Codex CLI with GPT-5.6-Terra) did each task twice, once with ordinary files and a shell and once through Solace only, and hidden tests graded the results. A step is one round trip to the model.

- All 20 attempts passed.
- Through Solace, the agent used 10% fewer tokens and 17% fewer steps overall.
- Per task, results ranged from 46% fewer tokens to 37% more. Resuming work from a saved note used 35% fewer.

- The first edit came after a median of 5.5 steps through Solace and 5 with files. The target is 3.

Each task ran once, and repeat runs vary by about 15%, so these are early signs rather than settled numbers.

8. The Solace language (planned)

The prototype stores Gleam code. The next step replaces Gleam with a language of Solace's own, designed so that what an agent can't see in ordinary code is written into the program, where the compiler can check it.

- **Declared effects.** A function can affect the outside world only through a value it was handed, such as a `Clock`, `Database` or `Http`. Its signature lists the effects it uses, and the compiler checks the list. A function without such parameters is pure. Tests pass fakes, like a fixed clock or seeded random numbers, so they behave the same on every run. A component given only `Clock` and `Log` cannot reach the network or disk.
- **Contracts and examples in the signature.** Preconditions, postconditions and examples sit beside the function, written in ordinary expressions. Contracts are checked at compile time where possible and at run time otherwise; examples run as tests.
- **Types that rule out wrong meanings.** There is no null (absence is `Option`) and there are no exceptions (failure is `Result`). Distinct types stop an `AccountId` being passed as an `OrderId`, refinements such as `Int where 0 <= it and it <= 100` restrict values, and units of measure stop meters being added to seconds.
- **State machines as a construct.** The compiler checks that every transition is handled and that finished states have no way out.
- **Explicit unfinished code.** A typed hole, `?rate "Flat rate or per-country table?"`, marks a gap. The rest of the program still compiles and runs its tests, a test that reaches the hole fails as unfinished rather than wrong, and the question goes to a human.
- **A small language.** No inheritance, overloading, implicit conversions, macros, reflection or global mutable state, and one way to write each construct, so an agent can read the whole reference at the start of a session.

Solace compiles to Erlang source for services on the BEAM (Erlang's virtual machine) and to JavaScript for browser interfaces.

9. What's next

- **Now:** repeat the measurements and test the task-based starting list and the smaller edit command.
- **Next:** build the Solace language (section 8). The milestone is agents maintaining the test project through Solace alone, typically reaching the first edit within 3 steps.
- **Later:** conflict detection between concurrent changes, a view for humans showing contracts, behavior and pending decisions, and exact record-and-replay of program runs.